MTA

AUDIO EDITOR

VST3 Manual

MTA Audio Editor
0.3.0-r210
Build: 20261015171900
Alessandro De Salvo
License: EUPL-1.2



Windows macOS Linux Android iOS

VST3 manual - MTA Audio Editor 0.3.0-r210

Server DSP engine

The C++ engine and VST3 audio effects are also available on the server for previews, bounces, exports and live browser playback. An administrator selects C++ in Settings > Web App. This preference is separate from native application settings; existing sessions retain their engine. RPM/DEB and Docker include the Pedalboard/PyAV runtime and compile the DSP at build time, never at startup. The Current engine remains available for offline rendering.

Installing and using server VST3 effects

The administrator installs Linux plugins matching the server CPU architecture in /opt/mta/vst3, /usr/lib/vst3 or /usr/local/lib/vst3. MTA_VST3_PATHS accepts additional colon-separated directories. Roots must be outside project storage and not writable by group/other users. For Docker, mount the plugin directory read-only at /opt/mta/vst3. Third-party plugins are not bundled; the runtime and plugins have separate licenses.

In the mixer, choose VST3 (server), select an effect and adjust its parameters with the configuration button. Track and master effects retain their order; input gain, faders, pan, mute/solo and master fades remain active. Parameters are saved in the project and reused for export. Original plugin editor shows the authentic plugin UI in the browser with mouse and keyboard input. The server package includes a private Xvfb built for EL10 too, and uses libXtst on a private authenticated display and Linux Landlock filesystem confinement, ABI 3 or newer. A missing kernel/container capability is reported explicitly. Plugin file choosers must not expose other users' files or server secrets. No server sound device is opened.

Live server playback

With C++ selected and at least one active VST3 insert or MIDI track, Play prepares the sources and starts a separate worker for this user session. Effect instances remain alive as small stereo PCM blocks are sent to the authenticated browser. The cursor follows audio actually heard, not already-produced audio. Pause, resume and seek discard future buffers and restart at the right position; seeking retains parameters and resets DSP history. Timeline edits require source preparation again.

Network buffering adds latency: normally up to about 1.4 seconds of audio are queued, plus network/device delay and effect lookahead. This is not low-latency input monitoring. On slow connections the cursor waits for audio and a buffering notice appears. A failed/hung worker stops the session without substituting dry audio. Each user may hold two sessions; the global default is eight, and abandoned sessions are removed after 60 seconds. Playback tails are bounded to two seconds. Acceptance on the actual server and with the user's own plugins remains necessary.

Live playback in native applications

Select the C++ audio engine in application Settings. VST3 audio effects on tracks and master then process small blocks during playback. The Current engine remains available and uses rendered effects. Export remains an offline render.

C++ manages the default system output device. An isolated worker keeps VST3 instances alive through Pedalboard, whose native processor uses JUCE. Built-in filters run through PyAV/FFmpeg. The device callback consumes a ring buffer without Python, plugin calls or waits. Plugins do not execute directly inside the hardware callback.

Installation and discovery

Native builds install the runtime from requirements-vst3.txt: Pedalboard, PyAV and Mido. Install plugins separately under their own licences and with the same CPU architecture as the application. MTA does not bundle commercial plugins. Standard Windows, macOS and Linux VST3 directories are supported. MTA_VST3_PATHS adds directories using the operating system path separator.

Inserts and parameters

Choose Insert and Add VST3 on a mixer channel or the master. Built-in and VST3 inserts retain their selected order, including mixed chains. Track input gain precedes inserts; track fader and pan follow them. Master inserts precede master fader and fades. Master-bypass bounce tracks are not processed through the master twice.

With C++ selected, normalized 0-1 parameter sliders update the running worker without reloading the plugin. The output queue normally holds up to three 512-sample blocks (about 32 ms at 48 kHz), plus device latency and effect lookahead. This is not a guarantee of total latency. Changing chain structure or built-in filters can briefly pause playback while rebuilding the graph.

Original editor and persistence

Original plugin editor opens the UI of the same instance used by live processing. Audio continues while it is open. Opening it before playback prepares a paused session. Closing it saves normalized parameters and opaque plugin state. Before stopping the worker, the application attempts a final state snapshot into the project. A hung plugin may prevent recovery of the latest edits; the previous saved state remains available.

Projects retain settings and paths, not plugin binaries. Reopening a project requires the compatible plugin to be installed. One live editor window is allowed at a time. Close the current window before adding or removing plugins from its graph.

Transport, alignment and errors

Play, pause, seek and completion use the native clock. Seeking resets DSP memory and starts at the cursor while retaining parameter settings; previous reverb or delay tails are not reconstructed. Each route pulls compensated effect output before summing, preventing a buffered plugin from shifting only its track. Lookahead without output is bounded to 960,000 input samples; exceeding it reports an error. Playback tails after the project are bounded to two seconds.

Worker crashes, non-finite audio and timeouts stop the session and report an error rather than silently bypassing effects. The output device lives outside the application process. Restart playback or select Current engine after a failure. Underrun counters indicate that the worker could not supply audio quickly enough; reduce effect load or use Current engine.

VST instruments and MIDI

Use the MIDI sidebar to create/import independent MIDI tracks. SMF0/1/2 up to 8 MiB and PPQ/SMPTE timebases are supported. SMF2 retains independent tempo maps; SMF0/1 preserve event timing with the project grid. Existing audio is not retimed.

Double-click the piano roll to add notes; drag or resize their right edge with snap. Selected notes expose pitch, start/duration in milliseconds, velocity and channel. Save MIDI persists edits; Save and play project saves and starts playback. Double-click a MIDI track or use its context menu to reopen the editor. Instrument parameters and the original editor are available both natively and on the server. When monitoring is active, the server editor uses the same audition-bus instance.

The instrument precedes gain, inserts and mixer controls. Notes/controllers remain editable after reopening; audio bounce/export includes the instrument. SMF export contains events and current BPM/time signature, not plugin binaries, presets or audio. Seeking resets voices and restores held notes, sustain and controllers without reconstructing earlier oscillator phase or tails. Clip boundaries release sustain and notes.

Record MIDI input captures notes/controllers using Web MIDI or native RtMidi. Connect MIDI devices, select input/output and choose VSTi, hardware or both. Start monitoring auditions the VSTi while the project is stopped; the on-screen keyboard needs no hardware. Panic releases all voices. Native monitoring uses the C++ output; server monitoring uses a separate authenticated low-buffer WebSocket stream and displays network/buffer latency. To include external MIDI sound, use the red Record external audio button and select the connected audio or loopback input. Latency compensation and optional project playback are available. Recording is published and saved as a track before completion is reported; exports then include that track. Optional MIDI Clock sends 24 PPQ, Start/Continue/Stop and Song Position. Port selections are DB preferences scoped to the user/project. Recording remains a draft from source time zero. The MPE / SysEx / MIDI 2.0 panel provides lower/upper MPE zones, member-channel allocation and pitch-bend RPN setup. Complete SysEx packets survive SMF import/export; hardware I/O requires explicit consent and seeking never replays them. Native MIDI 2.0 uses libremidi and retains original UMP words. Browser Web MIDI remains MIDI 1.0; UMP events can be imported/exported as lossless MTA UMP JSON. The VST3 bridge preserves 16-bit velocity, 32-bit pressure/controller values and mapped note expressions. UMP types without a VST3 mapping are explicitly rejected for instrument rendering but remain usable through raw hardware UMP output. SMF type 2 and SMPTE are supported. Limits: 50,000 notes, 100,000 controllers, 150,000 expanded events and 24 hours.

The compiled host synchronizes VST instruments/effects to decimal BPM, metronome time signature and tempo maps, samples, seconds, PPQ, bar start and play/pause state. This also applies to export. The runtime is built during packaging, never at startup. Adaptive and zoned maps update BPM/meter while retaining continuous PPQ; processing blocks split at tempo changes and MIDI Clock follows the same map. Loop flags describe VST context and do not themselves enable transport looping. See docs/VSTI_MIDI_R210.md for timing and hardware limitations.

Scope and verification

The host supports mono/stereo audio effects with up to 64 tracks. VST instruments/MIDI are supported in the shared graph. Plugins with auxiliary inputs expose External sidechain: select source track, bus and gain. The tap follows input gain and precedes inserts/fader, including muted source tracks. Live playback, bounce and export use the same C++ graph. Client audio capture uses a native C++ ring or browser AudioWorklet, bounded uploads, quota checks and transactional import. Dedicated ASIO device selection remains outside scope. The older experimental Steinberg probe remains a separate diagnostic path, not the backend of this live host.

Tests use a real CHOWTapeModel VST3 from the official Pedalboard test repository, pinned by commit and SHA-256 and not redistributed in this package. They cover persistent instances, parameter changes, state, master effects, seek and shutdown. CI configures a virtual display for the original editor. Listening tests on Windows/macOS and acceptance with the user's own plugins are still required.

Optional General MIDI instrument

FluidSynth with the reviewed FluidR3 GM bank can be selected per track. Configure program, bank, gain, polyphony and reverb in instrument parameters. Install the optional library/bank separately and preserve their licenses. MIDI remains editable when they are missing. See AI_STUDIOS_IMPLEMENTATION_R210_EN for exact checksums and restrictions. The full AI studios and dedicated ASIO/hardware routing remain open.

Licenses and redistribution

- MTA Audio Editor: EUPL-1.2.
- Steinberg VST3 SDK (MIT-licensed releases): MIT.
- Pedalboard (optional VST3 backend): GPL-3.0.
- X.Org server (private framebuffer): MIT/X11; XSERVER-COPYING licence included in the package.



Mixer and FX

Technical appendices

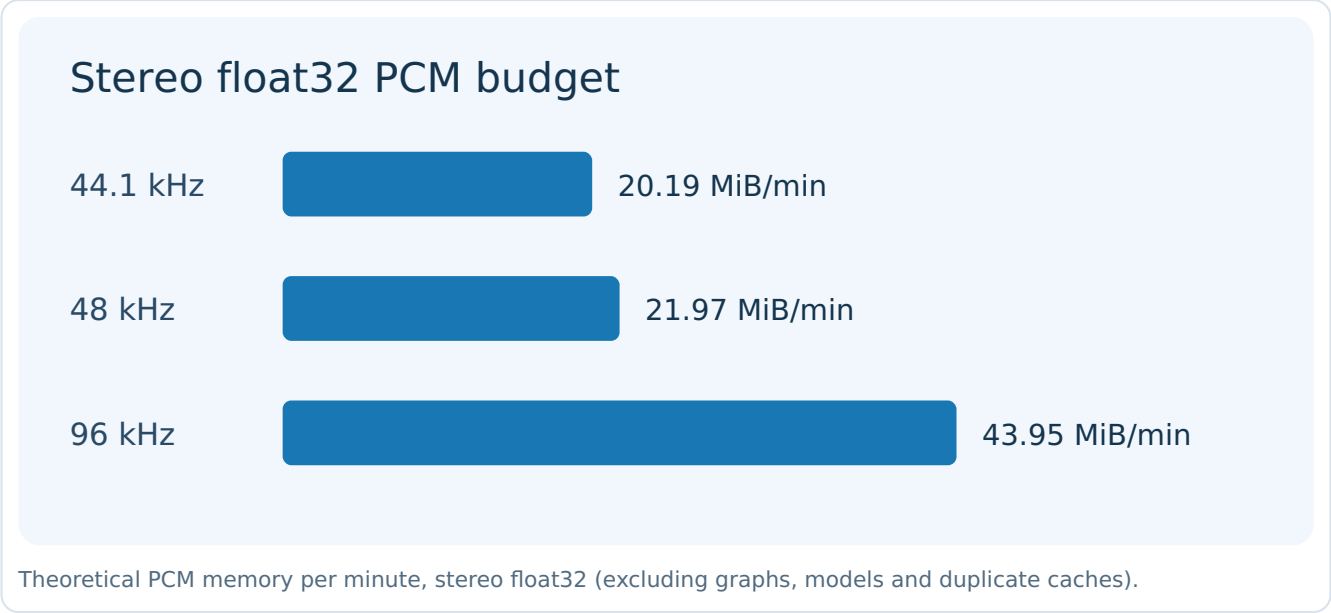
Documented state: 0.3.0-r210. Sizing guidance is an operational estimate, not a certified minimum. Local results do not certify physical devices, plugins or production servers.

A. Application types and capabilities

Type	Audio / DSP	AI	Plugin / MIDI
Web + server	Browser Web Audio; server C++ DSP	Global server queue; tokens for remote clients	Server Linux VST3, remote editor; Web MIDI where supported
Windows desktop	Current or C++; device transport	Local engines or token-authenticated remote server	Compatible Windows VST3; native MIDI
macOS desktop	Intel / Apple Silicon; Current or C++	Local models or server; accelerator support depends on engine	Matching-architecture macOS plugins; native MIDI
Linux desktop	Current or C++; local device	CPU / CUDA / ROCm depending on runtime and hardware	Linux VST3; ALSA/MIDI subject to availability
Android / iOS / iPadOS	Mobile shell; browser/ WebView audio	Server; on-device paths require a compatible model/ runtime	Server-side plugins; browser MIDI depends on platform

B. Resources and sizing

Profile	CPU / RAM	Disk / network	Note
Browser	4-8 GiB device RAM: planning baseline	HTTPS; stable network for streams and remote editors	256 MiB PCM cache; no guarantee for obsolete WebViews
Desktop / DSP	4 cores / 8 GiB starting point; 16+ GiB for dense sessions	SSD recommended; allow sources, exports and models	Plugins, sample rate and channel count determine actual load
Server editor	4 vCPU / 8 GiB for small tests; size for concurrency	Persistent volume, backups, bandwidth for concurrent sessions	Not a guaranteed user capacity
Server / desktop AI	16-32+ GiB RAM initial budget; model dependent	Separate weights/runtimes: several GiB, sometimes tens	VRAM/unified memory and precision depend on model; validate each engine
Mobile	Browser memory/CPU; avoid oversized projects	Stable Wi-Fi recommended for audio and remote plugins	Server DSP does not become phone-local



Theoretical PCM memory per minute, stereo float32 (excluding graphs, models and duplicate caches).

```
PCM bytes = seconds × sample rate × channels × 4
```

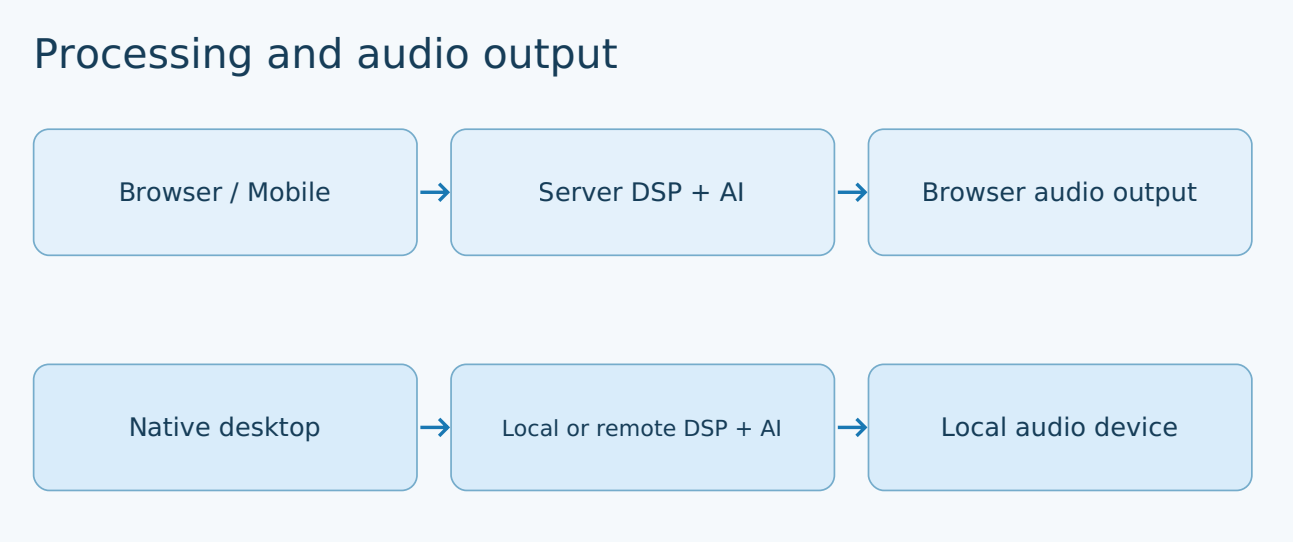
C. Limits and behavior

Area	Limit / behavior
Project	44.1 / 48 / 96 kHz; sample rate fixed after first track. Timeline up to 24 h.
Metronome	Standard, marker zones, adaptive. Pre-roll 0-64 bars; grid BPM 20-400 including decimals.
FX	Up to 16 inserts per chain; validate dependencies and VST3 compatibility per plugin.
MIDI	SMF0/1/2 up to 8 MiB; PPQ and SMPTE timebases. UMP/ MIDI 2.0 depends on driver/provider, not guaranteed for every device.
AI	Global queue; configurable artifact limits and timeouts. An installed model does not guarantee GPU compatibility.
Streaming C++	Adaptive buffer 0.35-1.2 s plus block; not a guarantee of professional low-latency monitoring.
Storage / billing	New users default to 5 GiB; admins have no default quota. Quotas/credits depend on plan and permissions.

D. Observed performance and validation

Measurements from the preceding local checkpoint: Chromium, eight 30-second stereo tracks; test machine and load do not represent the production server. Original reports remain in the project validation directory.

Test	Observed result	Local source
C++ playback	Start 1.239 s; fader response 0.538 s; 0 underruns	r210-continuous-pcm-browser.json
C++ jitter / UI stalls	Network delays up to 180 ms, five 450 ms UI stalls: 0 underruns	r210-continuous-pcm-stress.json
Current engine	Start 2.198 s; cached restart 0.265 s	r210-continuity-current-engine.json



Processing and audio output locations.

Certification still depends on real-platform checks: audio interface, MIDI driver, third-party plugins, GPU and proxy. Accepted consumption is accounted; source and validity remain in the audit; zero consumption does not prove no GPU use.